2_Lafi_Dohrendorf_Kauws

May 19, 2026

1 Machine Learning - Practical 2 - Introduction to Pytorch and Linear Regression with Pytorch

Names: {Lafi, Dohrendorf, Kauws}
Summer Term 2026

2 IMPORTANT SUBMISSION INSTRUCTIONS

You should work on the exercises in groups of 2-3. It is on you how you collaborate, but please make sure that everyone contributes equally and also that you understand all the solutions. You will be asked to present your group's solution in the tutorials and you should be well prepared to present any part of it.

- When you've completed the exercise, download the notebook and rename it to `<surname1>_<surname2>_<surname3>.ipynb`.
- Only submit the Jupyter Notebook (ipynb file). No other file is required. Upload it on `Stud.IP` -> `Machine learning 1` -> `Files` -> `Submission of Homework 1`.
- Make only one submission of the exercise per group.
- The deadline is strict.
- In addition to the submissions, every member of your group should be prepared to present the exercise in the tutorials.

Implementation - Do not change the cells which are marked as "DO NOT CHANGE", similarly write your solution to the marked cells.

2.1 How to work on the exercise?

Generally, for machine learning you often need access to a machine with a GPU. This is not strictly required for this homework but we recommend using Kaggle, which offers convenient access to a GPU and has all the dependencies that we need preinstalled (here are some initial steps to work with Kaggle Notebooks). You can load this notebook on kaggle via `File` -> `Import Noteboook` -> `Browse and  Import`. Alternatively, you can also use Colab.

3 Introduction

In this task you will get to know the basic tools used by the machine learning community. Later, we will build a linear regression model with PyTorch and perform training and prediction the

linear regression problem from the previous practical. The goal of this tutorial is to understand the PyTorch framework and getting to know to use it.

3.1 Tutorials

Some python libraries are required to accomplish the tasks assigned in this homework. If you feel like you need to follow a tutorial before, feel free to do so:

- PyTorch Tutorial
- Seaborn Tutorial (data visualization library on top of matplotlib)

[1]:
```
import random
import numpy as np
import matplotlib as mpl
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import torch
import pathlib
from torchvision import datasets, transforms
from torch.utils.data import DataLoader, sampler, random_split
```

[2]:
```
torch.set_default_dtype(torch.float64)
```

3.2 System checks

Perform some rudimentary system checks. Do we have a CUDA-capable device? Multiple? Is CuDNN active (huge speedups for some networks)?

[3]:
```
torch.cuda.is_available(), torch.backends.cudnn.is_available(), torch.cuda.
↪device_count()
```

[3]:
```
(True, True, 1)
```

If you see now that now that there is no CUDA-capable device available, you have to activate the GPU.

Click the top-right corner menu `|<` -> `Settings` -> `Accelerator` -> Select `GPU` as Hardware accelerator.

Now check the availability again (after re-running the imports)

[4]:
```
torch.cuda.is_available(), torch.backends.cudnn.is_available(), torch.cuda.
↪device_count(), torch.cuda.current_device()
```

[4]:
```
(True, True, 1, 0)
```

Choose your device for computation. CPU or one of your CUDA devices?

[5]:
```
# DO NOT CHANGE
# use_cuda = True
use_cuda = True
use_cuda = False if not use_cuda else torch.cuda.is_available()
device = torch.device('cuda:0' if use_cuda else 'cpu')
torch.cuda.get_device_name(device) if use_cuda else 'cpu'
print('Using device', device)
```

```
Using device cuda:0
```

4 PyTorch: Getting to know Tensors

feel free to skip this before 'Machine Learning with Pytorch' section if you feel confident enough
PyTorch is a library for machine learning on arbitrary datasets, including irregular input data such as graphs, point clouds and manifolds.

In this short tutorial we will explore some of its features to handle data in tensors. If you want, you can look into more detailed PyTorch tutorials online.

[6]:
```
# create a numpy array
numpyarray = np.arange(10).reshape(2, 5)
# convert to pytorch tensor
a = torch.from_numpy(numpyarray)
```

Let us find out what the properties of this tensor 'a' are.

[7]:
```
# TODO print the tensor
print(a)
```

```
tensor([[0, 1, 2, 3, 4],
        [5, 6, 7, 8, 9]], dtype=torch.int32)
```

[8]:
```
# TODO print its type
print(f"Datatype of tensor a: {a.dtype}")
```

```
Datatype of tensor a: torch.int32
```

[9]:
```
# TODO print its shape
print(f"Shape of tensor a: {a.shape}") # Der Tensor hat 2 Zeilen/rows und 5␣
 ↪Spalten/columns. Alternativ a.size()
```

```
Shape of tensor a: torch.Size([2, 5])
```

[10]:
```
# TODO print its size (number of elements/number of Bytes) <= ?
print(f"Size of tensor a: {a.numel()} Elements")
print(f"Size of tensor a: {a.element_size() * a.numel()} Bytes") # in einem␣
 ↪Tensor genau wie in einem numpy array sind alle elemente gleich groß. tensor.
 ↪element_size() gibt die Größe eines Elements im Tensor in Bytes aus und␣
 ↪tensor.numel() die Anzahl an Elementen im Tensor. Das Produkt ist dann die␣
 ↪größe des ganzen Tensors
```

```
Size of tensor a: 10 Elements
Size of tensor a: 40 Bytes
```

[11]:
```
# TODO create a new numpy array out of the tensor and print its size
array = a.numpy()
print(f"Size of array: {array.size} Elements")
print(f"Size of array: {array.nbytes} Bytes")
```

```
Size of array: 10 Elements
Size of array: 40 Bytes
```

Let's create some new tensors.

[12]:
```
# TODO create a tensor of shape (2,5) filled with ones of type int and print it
ones_tensor = torch.ones(size = (2,5), dtype = int)
print(ones_tensor)
```

```
tensor([[1, 1, 1, 1, 1],
        [1, 1, 1, 1, 1]])
```

[13]:
```
# TODO create a tensor of shape (3,4) filled with zeros and print it
zeros_tensor = torch.zeros((3,4))
print(zeros_tensor)
```

```
tensor([[0., 0., 0., 0.],
        [0., 0., 0., 0.],
        [0., 0., 0., 0.]])
```

[14]:
```
# TODO transpose the tensor 'a'
a.t()
```

[14]:
```
tensor([[0, 5],
        [1, 6],
        [2, 7],
        [3, 8],
        [4, 9]], dtype=torch.int32)
```

Now change a value in the numpy array. Does the corresponding tensor change?

[15]:
```
# TODO change value in numpy array and inspect the tensor a
numpyarray[0,0] = 10 # Ersezten des Elements in der ersten Zeile der ersten␣
 ↪Spalte mit 10


print(a)
print(array)
```

```
tensor([[10,  1,  2,  3,  4],
        [ 5,  6,  7,  8,  9]], dtype=torch.int32)
[[10  1  2  3  4]
 [ 5  6  7  8  9]]
```

The value in the tensor changes too

Does it work the other way round as well?

Yes the value in the first row in the first colomn of the array "array" which we made of the tensor a which we made out of the array numparray which we changed changed too

Now we want to make use of the different devices available, namely cpu and gpu.

[16]:
```
# TODO move 'a' to the gpu
if torch.cuda.is_available():
    a = a.to("cuda") # cuda:0 , cuda:1, ... wenn mehrere GPUs

print(a.device)

numpyarray[0,0] = 20
print(a)
```

```
cuda:0
tensor([[10,  1,  2,  3,  4],
        [ 5,  6,  7,  8,  9]], device='cuda:0', dtype=torch.int32)
```

If you change a value in the tensor 'a' now, does the corresponding value in the tensor on the GPU change as well?

No

5 Machine Learning with Pytorch

The process of training and evaluating a machine learning model begins with **data loading**. A dataset needs to be chosen on which the model should be trained. This data might need some preprocessing (like resizing or normalizing) of images. As Pytorch does not have in-build preprocessing for data panels we have to define our custom normalization inside a Dataset or Dataloader. Splitting of the data in different sets is necessary. We need a set to train on, a set to validate the training progress and a set to test the model after training.

The next step is to **specify the model and its optimizer**, as well as the loss function. An important hyperparameter is the learning rate which influences how big the changes of the parameters should be after calculating the loss.

The **model fitting** is split into training and evaluation. In the training process the model does a forward pass which means the data is presented to the model and the model outputs a prediction. The loss compares the prediction with the ground truth. In the backward pass the gradient with regard to the parameter is calculated and the parameter are updated by the gradient step. In the evaluation process the loss is computed on the entire validation set. This is done to see how well the model operates on data for which the params were not optimized before in order to avoid overfitting. The model fitting process is repeated for `N` epochs which is another hyperparameter that needs to be chosen carefully.

After the training we evaluate the final model on the test set.

We'll go through the individual steps in the course of this notebook using linear regression as an example.

5.1 Data Loading and Preprocessing

5.1.1 Training, Validation and Test Sets

For the correct fitting of a neural network model we need three dataset components: one for training, one for validation in the training process, and the last one for testing the results of the training on unseen data.

Note that you should not use the test set in any part of your training and model selection procedure. It should be only used for showing the final results.

Usually, train and test datasets are already split in the provided kaggle datasets but as we work with a custom dataset, we would have to make train-test split ourselves first.

[17]:
```
from sklearn.model_selection import train_test_split
```

[18]:
```
test_size = 0.2
target_clm='critical_temp'
```

[19]:
```
# TODO: load data - same as in the previous practical and make train_test_split␣
↪from sklearn
data = pd.read_csv("superconduct_train.csv")

train, test = train_test_split(data, test_size = test_size, random_state = 1) #␣
↪test_size := how much of the data is for testing, random_state := for␣
↪reproducability (analogue to seed)
```

5.1.2 Dataset

PyTorch has 2 entities to load data. They are **Dataset** and **Dataloader**. **Dataset** is a class, which defines your data and often applies data preprocessing transformations, like normalization. It should have at least 3 functions : * **init** - as any other init. Usually, you would provide path to dataset here or dataset elements. * **len** - should return the whole dataset size * **getitem** - this function returns 1 pair of data and label, also here preprocessing transformations are usually applied

For the next exercise, take a look here for an example https://stanford.edu/~shervine/blog/pytorch-how-to-generate-data-parallel

[20]:
```
class Dataset(torch.utils.data.Dataset): # Klasse Dataset erbt (bestehende)␣
↪Klassenatribute und Methoden von torch.utils.data.Dataset bzw. in unserem␣
↪Fall zwingt es uns Objektattribute zu initalisieren (mit dem Konstruktor␣
↪__init__) und Methoden __len__ und __getitem__ zu definieren. Damit später␣
↪alles kompatibel ist.
```

```
  def __init__(self, df, target_clm, mean=None, std=None, normalise=True): #␣
↪Konstruktor, der Objektattribute initalisiert wobei self (erstes Argument␣
↪der Funktion das Objekt (Dataset) selber ist.
      '''
      TODO: save params to self attributes,
      x is data without target column
      y is target column
      transform df to_numpy
      '''
      self.normalise = normalise

      self.x = df.drop(columns=[target_clm]).to_numpy() # Alternativ: df.
↪drop(columns = [target_clm]).to_numpy() # Bei Features x keine unnötige␣
↪Dimension zu entfernen, da mehrere Spalten (anders als bei target y)
      self.y = df[target_clm].to_numpy().reshape(-1, 1) # Alternativ: np.
↪squeeze(df.loc[:, df.columns == target_clm].values) (unnötige Dimension zu␣
↪entfernen) # reshape damit 2 dimensionen [n,1] (später wichtig für input in␣
↪models)

      self.mean = mean if mean is not None else self.x.mean(axis = 0) # <=>␣
↪Mittelwert über erste Achse (Zeilen/vertikal) [über erste Achse (Zeilen)␣
↪aggregieren] (für features, da nur die normalisiert werden [siehe␣
↪vorgegebener Code weiter unten]) <=> Mittelwert für Spalten # (( Alternativ:␣
↪np.array([np.mean(df[:, row]) for row in rows]) )) <= nicht mehr wahr
      self.std = std if std is not None else self.x.std(axis = 0) # <=>␣
↪Standardabweichung über erste Achse (Zeilen/vertikal) (für features, da nur␣
↪die normalisiert werden [siehe vorgegebener Code weiter unten]) # ((␣
↪Alternativ: np.array(np.std(df[;, row]) for row in rows]) )) <= nicht mehr␣
↪wahr # Addition von 1e-8 damit keine Gefahr der Teilung durch 0 bei␣
↪Normalisierung
      self.std = np.where(self.std == 0, 1e-8, self.std) # verhindern, dass␣
↪später beim Normalsieren durch 0 geteilt wird

  def __len__(self):
      return len(self.x) # <=> Wieviele samples/Beobachtungen? <=> Wieviele␣
↪Zeilen?

  def __getitem__(self, index):
      x = self.x[index]
      y = self.y[index]

      # normalize if required
      if self.normalise:
          x = (x - self.mean) / self.std # Normalisierung von X bei Ausgabe␣
↪von samples
```

```
        # convert to tensors
        x = torch.tensor(x, dtype = torch.float32)
        y = torch.tensor(y, dtype = torch.float32)

        return x, y # Ausgabe 1 sample: X/features(normalisiert)/data und y/
↪target(nicht normalisiert)
```

[21]:
```
tmp_dataset = Dataset(train, target_clm, normalise=False)
```

[22]:
```
# TODO calculate the mean and standard deviation of the train dataset
mean = tmp_dataset.mean
std = tmp_dataset.std
```

[23]:
```
# TODO define new datasets with mean, std and normalise=True
conductor_train = Dataset(train, target_clm, mean = mean, std = std, normalise␣
↪= True)
conductor_test = Dataset(test, target_clm, mean = mean, std = std, normalise =␣
↪True)
```

We need to **split** the train dataset in two sets, one for training and one for validation. While the training set needs to be quite large, the validation set can be relatively small. Take 10 % of the dataset as validation set. Assign samples *randomly* to the training and validation set, using a fixed seed to ensure that train and test splits are same across different model runs.

In fact, the good practice is to fix a global random seed not only the generator seed for even better reproducibility with `torch.manual_seed(0)`. Machine learning models often involve random initialization of weights, augmentations, dropout layers, and other stochastic processes. Without fixing the random seed, each run of the model may produce slightly different results, making it challenging to reproduce specific results or debug issues.

[24]:
```
# TODO split the train dataset in conductor_train and conductor_val
torch.manual_seed(0)
val_size = 0.1

n_total = len(conductor_train)
n_val = round(val_size * n_total)
n_train = n_total - n_val
conductor_train, conductor_val = random_split(conductor_train, [n_train, n_val])
```

[25]:
```
batch_size = 256
```

To load the data for model training, we need to define the **dataloaders**. A dataloader represents a Python iterable over a dataset and draws mini batches with random samples. **Dataloader** calls ___getitem___ function from the Dataset and forms the batches.

Use the batch size as specified above. Make sure we get shuffled samples in batches.

[26]:
```
# TODO create dataloader for training, validation and test
```

```
train_dataloader = DataLoader(conductor_train, batch_size = batch_size, shuffle␣
 ↪= True)
val_dataloader = DataLoader(conductor_val, batch_size = batch_size, shuffle =␣
 ↪True)
test_dataloader = DataLoader(conductor_test, batch_size = batch_size, shuffle =␣
 ↪True)
```

Let's get a data point now to see what we're dealing with.

For this, you might want to check out how python's iterator protocol works. It's simple and will give you an important insight into python: https://wiki.python.org/moin/Iterator.

[27]:
```
# TODO get an element of the train_dataloader
data_iter = iter(train_dataloader)

# element: first batch [first batch_size samples] of train_dataloader/train␣
 ↪dataset
x_batch, y_batch = next(data_iter)
```

[28]:
```
# TODO print the dimensions of for elements from the previous step
print("X batch shape:", x_batch.shape)
print("Y batch shape:", y_batch.shape)
```

```
X batch shape: torch.Size([256, 81])
Y batch shape: torch.Size([256, 1])
```

x has size ([batchsize], 81) –> 256 elements/batches (or whatever you have defined in your data loader), 81 feature values.

y has size ([batchsize], 1) –> 256 elements/batches (again depends on your data loader config). There's one target value for each set of the features.

5.2 Specify Model & Optimizer

5.2.1 Specify a Model

The task is now to define a model to train on the data. In this simple example, we only need **one fully-connected layer** as defined in *torch.nn.Linear* that produces a predicted label for a specific training input row.

Before, we set some variables: - the input and output size of the linear layer - how long we want to train the model (number of epochs) and - the learning rate.

[29]:
```
epochs = 1
input_dim = 81
output_dim = 1
lr = 0.001
```

[30]:
```
class LinearRegression(torch.nn.Module):
    """
    Linear regression model inherits the torch.nn.Module
```

```
    which is the base class for all neural network modules.
    """
    def __init__(self, input_dim, output_dim): ## Wie funktionieren die␣
↪Layer(mini Modelle im Neuronalen Netz) konzeptionell (was für layer?␣
↪(linear, etc.) Aktivierung? Wieviel Inputs und Outputs?) => Hier nur ein␣
↪Layer: Einfache Lineare Regression [OLS]
        """ Initializes internal Module state. """
        super(LinearRegression, self).__init__()

        # TODO define linear layer for the model
        self.linear = torch.nn.Linear(input_dim, output_dim)

    def forward(self, x): ## Übergibt Daten an Layer "Runs data through layers"
        """ Defines the computation performed at every call. """
        # What are the dimensions of your input layer?
        # TODO flatten the input to a suitable size for the initial layer
        x = x.reshape(-1, input_dim)
        # TODO run the data through the layer
        outputs = self.linear(x)
        return outputs
```

5.2.2 Instantiate the Model

Let us instantiate the model and take a look at the inside. It is always a good idea to verify that the actual architecture is what you intended it to be. Especially, when you start to create layers dynamically it is great for inspection/verification/debugging.

[31]:
```
# TODO instantiate the model
model = LinearRegression(input_dim = input_dim, output_dim = output_dim).float()
```

Feed the model to the GPU if available.

[32]:
```
# TODO move model to device you specified above
model = model.to(device)
```

Put the model in training mode.

[33]:
```
# TODO put the model in train mode
model.train() ## wichtig für komplexere Sachen. Hier praktisch kein Unterschied
```

[33]:
```
LinearRegression(
  (linear): Linear(in_features=81, out_features=1, bias=True)
)
```

5.2.3 Define a Loss Function

Since we're dealing with regression problem, MSELoss is the canonical choice for the loss.

[34]:
```
# TODO define the loss function
loss_function = torch.nn.MSELoss()
```

5.3 Model Fitting

5.3.1 Train the Model

Everything is set for the model to train!

- In the forward pass, the prediction is made using the previously defined model on the elements of the dataloader.
- Then the loss (or error) needs to be computed by comparing the prediction to the actual label.
- In the backward pass, the model learns and updates its weights based on the current gradient.

5.3.2 First, let's do all of the steps manually, without using the optimizer

Hints: * define number of epochs to see the dynamic. You need to see the effect over several epochs but it should no be too long. * use learning rate defined above as `lr` * when doing parameters update - do it under `with torch.no_grad():`. This would disable the gradient computation for the operations under it. And we don't need gradients for updating the weights step. * you need to update the model parameters. See here for more details on how to access them * don't forget to track the learning (loss)

[35]:
```
## TODO do a simple for-loop to illustrate how the gradient update if done over␣
 ↪batches.
## Print loss values across epochs to compare with the PyTorch optimizers later

epochs = 5

for epoch in range(epochs):

    total_loss = 0

    for x_batch, y_batch in train_dataloader:

        x_batch = x_batch.to(device)
        y_batch = y_batch.to(device)

        # forward pass => calculate predictions with old weights
        y_pred = model(x_batch)

        # loss
        loss = loss_function(y_pred, y_batch)

        # backward pass
        loss.backward() # calculates the gradient of the weights ("how does the␣
 ↪loss change when the weight changes")
```

```
        # manual gradient update
        with torch.no_grad(): # update weights without tracking gradients
            for param in model.parameters():
                param -= lr * param.grad # changes weights with formula␣
↪new_weights = old_weights - learning_rate * gradient with every batch

        # clear gradients
        model.zero_grad()

        total_loss += loss.item() # total_loss := Sum of Loss over all batches

    print(f"Epoch {epoch+1:3d}/{epochs:<3d} | Loss = {total_loss:12.4f}")
```

```
Epoch   1/5   | Loss =  101912.1456
Epoch   2/5   | Loss =   79909.7291
Epoch   3/5   | Loss =   67716.1408
Epoch   4/5   | Loss =   58548.1622
Epoch   5/5   | Loss =   51414.9778
```

This was equivalent to SGD optimizer

5.3.3 Now let's do it in the pytorch style using the optimizer

The optimizer is the learning algorithm we use. In this case, we use Stochastic Gradient Descent (SGD). Redefine the model and initialize SGD optimizer

[36]:
```
# TODO Redefine the model and initialize SGD optimizer, write a train loop as␣
↪above and compare the loss values

model = LinearRegression(input_dim = input_dim, output_dim = output_dim).float()
model = model.to(device)

loss_function = torch.nn.MSELoss()
optimizer = torch.optim.SGD(model.parameters(), lr = lr) # Initialisierung des␣
↪optimizers mit parametern des models und learning rate

epochs = 15

for epoch in range(epochs):

    total_loss = 0

    for x_batch, y_batch in train_dataloader:

        x_batch = x_batch.to(device)
        y_batch = y_batch.to(device)

        # forward pass
```

```
        y_pred = model(x_batch)

        # loss
        loss = loss_function(y_pred, y_batch)

        # backward pass
        loss.backward()

        # update parameters (automatically via optimizer initialised with␣
↪learning rate and model)
        optimizer.step()

        # clear old gradients
        optimizer.zero_grad()

        total_loss += loss.item()

    print(f"Epoch {epoch+1:3d}/{epochs:<3d} | Loss = {total_loss:12.4f}")
```

```
Epoch   1/15  | Loss =  102586.6829
Epoch   2/15  | Loss =   80371.2817
Epoch   3/15  | Loss =   67977.0721
Epoch   4/15  | Loss =   58734.8060
Epoch   5/15  | Loss =   51571.7666
Epoch   6/15  | Loss =   45919.6740
Epoch   7/15  | Loss =   41444.3508
Epoch   8/15  | Loss =   37929.0531
Epoch   9/15  | Loss =   35100.5576
Epoch  10/15  | Loss =   32867.3045
Epoch  11/15  | Loss =   31075.5612
Epoch  12/15  | Loss =   29625.7373
Epoch  13/15  | Loss =   28476.0261
Epoch  14/15  | Loss =   27528.6029
Epoch  15/15  | Loss =   26757.0792
```

5.3.4 Make a Prediction

Now that our model is trained, we can make a new prediction by inputting an unseen data row from the test dataset.

Run this cell several times, does the model predict accurately?

Set the number of epochs to 15 during training and try again!

The prediction is not very accurate for this individual test sample. This is expected because the model is only a simple linear regression model and the relationship in the data is likely more complex. With more epochs the model improves overall, but single predictions can still be far away from the true value.

[37]:
```
model.eval() # puts model in evaluation mode

# TODO get a random element of the test dataloader
test_batches = list(test_dataloader) # list of batches from test_dataloader

x_batch, y_batch = random.choice(test_batches) # randomly draw one batch from␣
 ↪list

idx = random.randrange(len(x_batch)) # draw random number from range/length of␣
 ↪batch

x_sample = x_batch[idx].to(device) # draws sample from batch with drawn random␣
 ↪number and transfers it to device (where model is)
y_sample = y_batch[idx] # draws sample from batch with drawn random number

# TODO make a prediction
with torch.no_grad():
    y_pred = model(x_sample)

# print predicted label and given label
print("Predicted value:", y_pred.item())
print("True value:", y_sample.item())
```

```
Predicted value: 51.344383239746094
True value: 74.5
```

5.3.5 Track and Plot the Training and Validation error

What we have seen so far is the basic principle of training a model and making a prediction. But one might be interesting to see more about the training process, for instance how the training error evolves with time.

For this step, we are going to **refine the training process** and **add some important information saving for plotting**.

Create a plot using **seaborn** that contains both the losses on training set and the losses on the validation set for each epoch.

The plot should look similar to this:

Note: Do not forget to add title, axis labels and a legend! This applies in general, please keep in mind for future exercise sheets.

[38]:
```
# TODO refine the training function from above
# it should contain:
# - saving of losses
# - returning the mean loss
```

```
def train_model(model, train_dataloader, loss_function, optimizer, device):

    model.train() ##

    train_losses = []

    for x_batch, y_batch in train_dataloader:

        x_batch = x_batch.to(device)
        y_batch = y_batch.to(device)

        # forward pass
        y_pred = model(x_batch)

        # loss
        loss = loss_function(y_pred, y_batch)

        # backward pass
        loss.backward()

        # update parameters (automatically via optimizer initialised with␣
 ↪learning rate and model)
        optimizer.step()

        # clear old gradients
        optimizer.zero_grad()

        # save batch loss
        train_losses.append(loss.item())

    mean_train_loss = np.mean(train_losses)

    return mean_train_loss
```

[39]:
```
# TODO write a validation function that calculates the loss on the validation␣
 ↪set
# you can also combine it with the training function

def validate_model(model, val_dataloader, loss_function, device):

    model.eval()

    val_losses = []

    with torch.no_grad():
```

```
        for x_batch, y_batch in val_dataloader:

            x_batch = x_batch.to(device)
            y_batch = y_batch.to(device)

            y_pred = model(x_batch)

            loss = loss_function(y_pred, y_batch)

            val_losses.append(loss.item())

    mean_val_loss = np.mean(val_losses)

    return mean_val_loss
```

[40]:
```
# TODO write a run_training function that
# - calls the train and validate functions for each epoch
# - saves the train_losses, val_losses as arrays for each epoch

def run_training(model, train_dataloader, val_dataloader,
                 loss_function, optimizer, device, num_epochs):

    losses = []

    for epoch in range(num_epochs):

        mean_train_loss = train_model(model, train_dataloader, loss_function,␣
↪optimizer, device)
        mean_val_loss = validate_model(model, val_dataloader, loss_function,␣
↪device)

        losses.append({"epoch": epoch + 1, "mean_train_loss": mean_train_loss,␣
↪"mean_val_loss": mean_val_loss})
        print(f"Epoch {epoch+1:3d}/{num_epochs:<3d} | train_loss =␣
↪{mean_train_loss:10.4f} | val_loss = {mean_val_loss:10.4f}")

    return losses
```

[41]:
```
# TODO call the run_training function and run it for 10 epochs.
num_epochs = 10

model = LinearRegression(input_dim = input_dim, output_dim = output_dim).float()
model = model.to(device)

loss_function = torch.nn.MSELoss()
optimizer = torch.optim.SGD(model.parameters(), lr = lr)
```

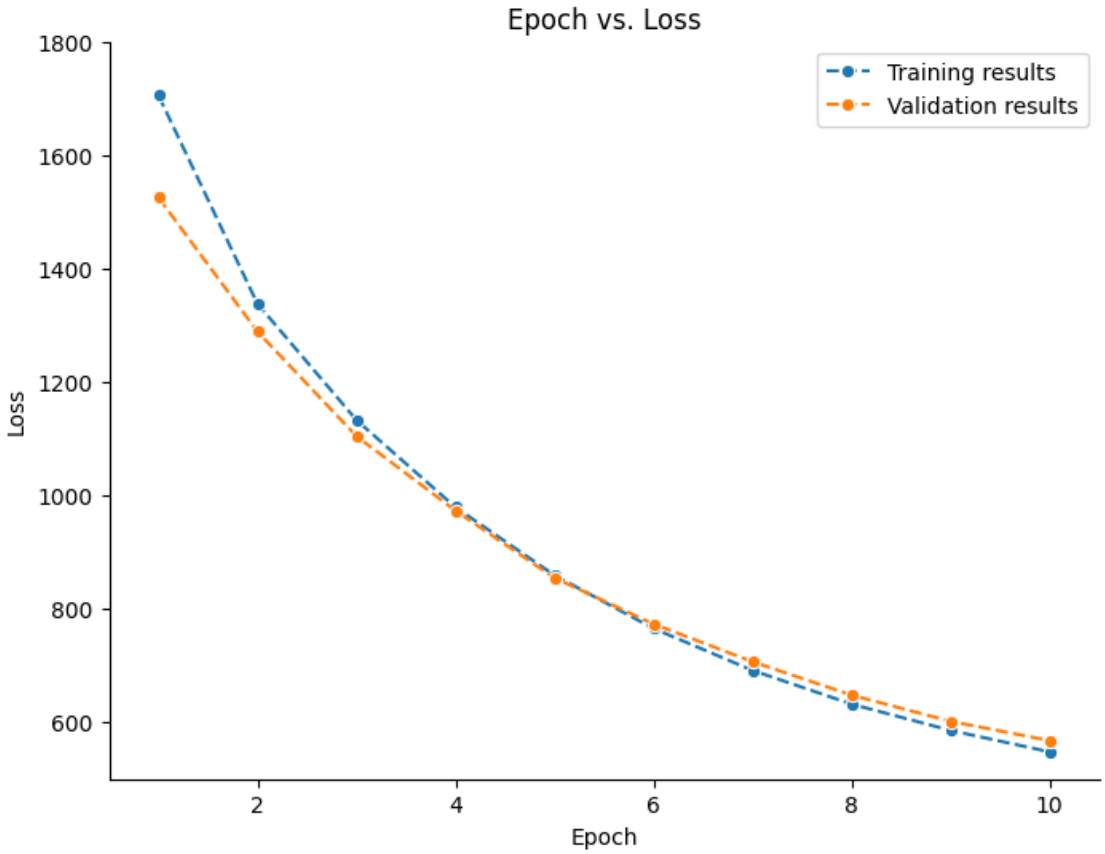
```
losses = run_training(model, train_dataloader, val_dataloader, loss_function,␣
 ↪optimizer, device, num_epochs)
```

```
Epoch   1/10  | train_loss =  1705.1062 | val_loss =  1525.1714
Epoch   2/10  | train_loss =  1337.7753 | val_loss =  1288.8011
Epoch   3/10  | train_loss =  1132.7887 | val_loss =  1104.4887
Epoch   4/10  | train_loss =   979.1624 | val_loss =   973.1227
Epoch   5/10  | train_loss =   859.1598 | val_loss =   854.5964
Epoch   6/10  | train_loss =   765.3786 | val_loss =   773.3222
Epoch   7/10  | train_loss =   691.4862 | val_loss =   706.8547
Epoch   8/10  | train_loss =   632.1029 | val_loss =   648.0013
Epoch   9/10  | train_loss =   585.6463 | val_loss =   601.6303
Epoch  10/10  | train_loss =   547.8786 | val_loss =   567.7747
```

[42]:
```
# TODO write a plot function

def plot_losses(losses):

    losses_df = pd.DataFrame(losses)

    plt.figure(figsize = (8, 6))

    sns.lineplot(
        data = losses_df,
        x = "epoch",
        y = "mean_train_loss",
        marker = "o",
        linestyle = "--",
        label = "Training results"
    )

    sns.lineplot(
        data = losses_df,
        x = "epoch",
        y = "mean_val_loss",
        marker = "o",
        linestyle = "--",
        label = "Validation results"
    )

    plt.xlim(0.5, 10.5)
    plt.ylim(500, 1800)

    plt.xticks(range(2, 11, 2))
    plt.yticks(range(600, 1801, 200))

    sns.despine()
```

```
    plt.title("Epoch vs. Loss")
    plt.xlabel("Epoch")
    plt.ylabel("Loss")
    plt.legend()
    plt.show()
```

[43]:
```
# TODO plot losses
plot_losses(losses)
```



Comment on the loss curve. Does it look as expected? Why/Why not? If not, what might be the reason.

The loss curve looks mostly as expected. The training loss decreases over time, which shows that the model is learning. The validation loss also decreases, but it is slightly noisier because it is measured on unseen validation data and the model is trained with stochastic mini-batches.

6 Hyperparameters influence

Now, once we successfully reproduced linear regression using PyTorch, lets explore the hyperparameters influence, such as learning rate or batch size.

Train several models with 30 train epochs and using different learning rates - [0.0001, 0.001, 0.01, 0.1, 1, 10]. What do you notice? Why?

Hints: * Do not forget to reinitialize models and update the optimizers * Use different colors and line styles to display different learning rates and train-validation splits

The learning rate has a strong influence on training. Very small learning rates learn slowly, while moderate learning rates converge faster. If the learning rate is too large, the optimization becomes unstable and the loss can diverge or become NaN (as it is the case for learning rate>=0.1 in our example).

[44]:
```
num_epochs = 30
## TODO - train models with different learning rates

learning_rates = [0.0001, 0.001, 0.01, 0.1, 1, 10]

all_losses = []

for lr in learning_rates:

    print(f"\nTraining model with lr = {lr}")

    model = LinearRegression(input_dim = input_dim, output_dim = output_dim).
 ↪float()
    model = model.to(device)

    loss_function = torch.nn.MSELoss()
    optimizer = torch.optim.SGD(model.parameters(), lr = lr)

    losses = run_training(model, train_dataloader, val_dataloader,␣
 ↪loss_function, optimizer, device, num_epochs)

    for loss in losses:
        loss["lr"] = lr
        all_losses.append(loss)
```

```
Training model with lr = 0.0001
Epoch   1/30  | train_loss =  2164.7234 | val_loss =  2131.5389
Epoch   2/30  | train_loss =  1931.2552 | val_loss =  1934.7289
Epoch   3/30  | train_loss =  1801.4132 | val_loss =  1844.3288
Epoch   4/30  | train_loss =  1718.9625 | val_loss =  1746.4312
Epoch   5/30  | train_loss =  1660.9044 | val_loss =  1702.9071
Epoch   6/30  | train_loss =  1614.3545 | val_loss =  1657.1925
Epoch   7/30  | train_loss =  1574.6581 | val_loss =  1609.1272
```

```
Epoch   8/30  | train_loss =  1539.8877 | val_loss =  1578.0470
Epoch   9/30  | train_loss =  1506.0652 | val_loss =  1544.8842
Epoch  10/30  | train_loss =  1476.8230 | val_loss =  1521.1051
Epoch  11/30  | train_loss =  1446.8069 | val_loss =  1495.5282
Epoch  12/30  | train_loss =  1419.1610 | val_loss =  1467.0713
Epoch  13/30  | train_loss =  1392.7603 | val_loss =  1430.2602
Epoch  14/30  | train_loss =  1366.9707 | val_loss =  1401.7248
Epoch  15/30  | train_loss =  1342.9965 | val_loss =  1378.3778
Epoch  16/30  | train_loss =  1318.9192 | val_loss =  1368.5298
Epoch  17/30  | train_loss =  1296.4499 | val_loss =  1341.1026
Epoch  18/30  | train_loss =  1274.6051 | val_loss =  1321.1938
Epoch  19/30  | train_loss =  1252.9722 | val_loss =  1305.1185
Epoch  20/30  | train_loss =  1231.8752 | val_loss =  1284.6931
Epoch  21/30  | train_loss =  1211.7503 | val_loss =  1259.1579
Epoch  22/30  | train_loss =  1192.8962 | val_loss =  1244.1692
Epoch  23/30  | train_loss =  1173.5790 | val_loss =  1212.8063
Epoch  24/30  | train_loss =  1154.9646 | val_loss =  1198.4051
Epoch  25/30  | train_loss =  1137.0139 | val_loss =  1178.9032
Epoch  26/30  | train_loss =  1120.2670 | val_loss =  1174.2432
Epoch  27/30  | train_loss =  1103.0798 | val_loss =  1159.4478
Epoch  28/30  | train_loss =  1086.3802 | val_loss =  1126.6061
Epoch  29/30  | train_loss =  1070.0140 | val_loss =  1113.2751
Epoch  30/30  | train_loss =  1054.7130 | val_loss =  1089.6520

Training model with lr = 0.001
Epoch   1/30  | train_loss =  1712.7972 | val_loss =  1530.6372
Epoch   2/30  | train_loss =  1340.3711 | val_loss =  1283.4260
Epoch   3/30  | train_loss =  1134.6104 | val_loss =  1103.9550
Epoch   4/30  | train_loss =   980.4411 | val_loss =   964.0876
Epoch   5/30  | train_loss =   859.9130 | val_loss =   855.8654
Epoch   6/30  | train_loss =   765.9755 | val_loss =   771.5733
Epoch   7/30  | train_loss =   691.5964 | val_loss =   698.9024
Epoch   8/30  | train_loss =   632.7715 | val_loss =   652.4913
Epoch   9/30  | train_loss =   586.2692 | val_loss =   603.8060
Epoch  10/30  | train_loss =   548.4950 | val_loss =   574.7808
Epoch  11/30  | train_loss =   518.5102 | val_loss =   544.7445
Epoch  12/30  | train_loss =   494.3814 | val_loss =   519.8188
Epoch  13/30  | train_loss =   475.3085 | val_loss =   502.1421
Epoch  14/30  | train_loss =   459.6649 | val_loss =   490.6282
Epoch  15/30  | train_loss =   446.4985 | val_loss =   478.3059
Epoch  16/30  | train_loss =   436.0627 | val_loss =   465.5929
Epoch  17/30  | train_loss =   427.0921 | val_loss =   453.2393
Epoch  18/30  | train_loss =   420.3677 | val_loss =   456.2940
Epoch  19/30  | train_loss =   413.8110 | val_loss =   441.2130
Epoch  20/30  | train_loss =   409.0881 | val_loss =   433.3718
Epoch  21/30  | train_loss =   404.5733 | val_loss =   434.4600
Epoch  22/30  | train_loss =   400.9919 | val_loss =   427.0104
Epoch  23/30  | train_loss =   397.7545 | val_loss =   419.7330
```

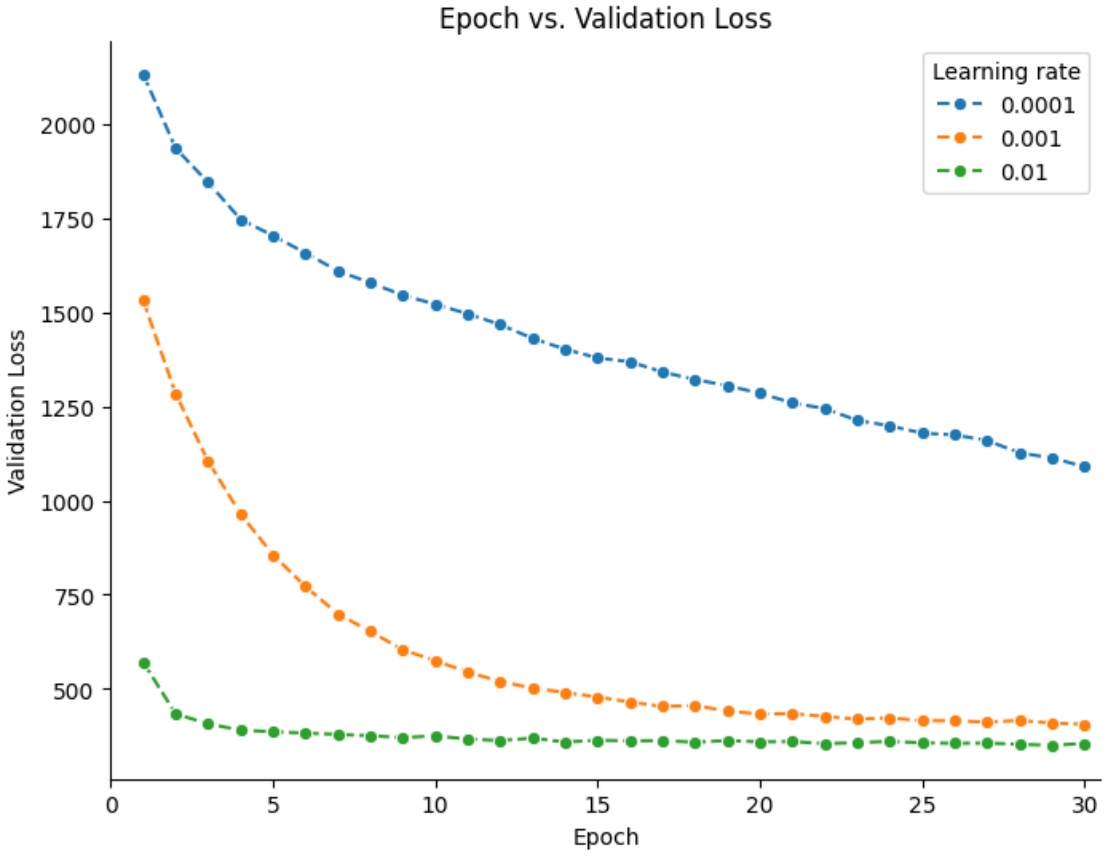
```
Epoch  24/30  | train_loss =   394.7544 | val_loss =   423.1355
Epoch  25/30  | train_loss =   392.3964 | val_loss =   415.7943
Epoch  26/30  | train_loss =   389.9454 | val_loss =   415.5126
Epoch  27/30  | train_loss =   387.9447 | val_loss =   412.1058
Epoch  28/30  | train_loss =   385.9665 | val_loss =   416.1876
Epoch  29/30  | train_loss =   384.3638 | val_loss =   409.5992
Epoch  30/30  | train_loss =   382.9481 | val_loss =   405.9817

Training model with lr = 0.01
Epoch   1/30  | train_loss =   934.7456 | val_loss =   570.9176
Epoch   2/30  | train_loss =   451.3203 | val_loss =   433.6500
Epoch   3/30  | train_loss =   393.4886 | val_loss =   407.6025
Epoch   4/30  | train_loss =   377.4306 | val_loss =   390.4409
Epoch   5/30  | train_loss =   369.3404 | val_loss =   386.4368
Epoch   6/30  | train_loss =   363.9464 | val_loss =   382.8677
Epoch   7/30  | train_loss =   359.9090 | val_loss =   379.3174
Epoch   8/30  | train_loss =   356.9332 | val_loss =   375.7396
Epoch   9/30  | train_loss =   354.6194 | val_loss =   370.7395
Epoch  10/30  | train_loss =   352.5214 | val_loss =   375.5061
Epoch  11/30  | train_loss =   351.1216 | val_loss =   367.0644
Epoch  12/30  | train_loss =   350.1677 | val_loss =   362.8613
Epoch  13/30  | train_loss =   348.3419 | val_loss =   369.4759
Epoch  14/30  | train_loss =   347.4798 | val_loss =   358.9985
Epoch  15/30  | train_loss =   346.2708 | val_loss =   363.9977
Epoch  16/30  | train_loss =   345.0716 | val_loss =   362.1564
Epoch  17/30  | train_loss =   345.2137 | val_loss =   363.1060
Epoch  18/30  | train_loss =   344.1537 | val_loss =   358.4625
Epoch  19/30  | train_loss =   343.1405 | val_loss =   363.0803
Epoch  20/30  | train_loss =   342.5038 | val_loss =   359.5510
Epoch  21/30  | train_loss =   341.5147 | val_loss =   360.5295
Epoch  22/30  | train_loss =   341.1992 | val_loss =   354.7779
Epoch  23/30  | train_loss =   341.0137 | val_loss =   357.5660
Epoch  24/30  | train_loss =   339.9865 | val_loss =   360.8747
Epoch  25/30  | train_loss =   339.4594 | val_loss =   357.0408
Epoch  26/30  | train_loss =   339.1113 | val_loss =   355.6135
Epoch  27/30  | train_loss =   338.1480 | val_loss =   356.2756
Epoch  28/30  | train_loss =   337.8428 | val_loss =   352.9800
Epoch  29/30  | train_loss =   337.1635 | val_loss =   350.3909
Epoch  30/30  | train_loss =   336.6839 | val_loss =   355.2535

Training model with lr = 0.1
Epoch   1/30  | train_loss =       nan | val_loss =       nan
Epoch   2/30  | train_loss =       nan | val_loss =       nan
Epoch   3/30  | train_loss =       nan | val_loss =       nan
Epoch   4/30  | train_loss =       nan | val_loss =       nan
Epoch   5/30  | train_loss =       nan | val_loss =       nan
Epoch   6/30  | train_loss =       nan | val_loss =       nan
Epoch   7/30  | train_loss =       nan | val_loss =       nan
```

```
Epoch   8/30  | train_loss =       nan | val_loss =       nan
Epoch   9/30  | train_loss =       nan | val_loss =       nan
Epoch  10/30  | train_loss =       nan | val_loss =       nan
Epoch  11/30  | train_loss =       nan | val_loss =       nan
Epoch  12/30  | train_loss =       nan | val_loss =       nan
Epoch  13/30  | train_loss =       nan | val_loss =       nan
Epoch  14/30  | train_loss =       nan | val_loss =       nan
Epoch  15/30  | train_loss =       nan | val_loss =       nan
Epoch  16/30  | train_loss =       nan | val_loss =       nan
Epoch  17/30  | train_loss =       nan | val_loss =       nan
Epoch  18/30  | train_loss =       nan | val_loss =       nan
Epoch  19/30  | train_loss =       nan | val_loss =       nan
Epoch  20/30  | train_loss =       nan | val_loss =       nan
Epoch  21/30  | train_loss =       nan | val_loss =       nan
Epoch  22/30  | train_loss =       nan | val_loss =       nan
Epoch  23/30  | train_loss =       nan | val_loss =       nan
Epoch  24/30  | train_loss =       nan | val_loss =       nan
Epoch  25/30  | train_loss =       nan | val_loss =       nan
Epoch  26/30  | train_loss =       nan | val_loss =       nan
Epoch  27/30  | train_loss =       nan | val_loss =       nan
Epoch  28/30  | train_loss =       nan | val_loss =       nan
Epoch  29/30  | train_loss =       nan | val_loss =       nan
Epoch  30/30  | train_loss =       nan | val_loss =       nan

Training model with lr = 1
Epoch   1/30  | train_loss =       nan | val_loss =       nan
Epoch   2/30  | train_loss =       nan | val_loss =       nan
Epoch   3/30  | train_loss =       nan | val_loss =       nan
Epoch   4/30  | train_loss =       nan | val_loss =       nan
Epoch   5/30  | train_loss =       nan | val_loss =       nan
Epoch   6/30  | train_loss =       nan | val_loss =       nan
Epoch   7/30  | train_loss =       nan | val_loss =       nan
Epoch   8/30  | train_loss =       nan | val_loss =       nan
Epoch   9/30  | train_loss =       nan | val_loss =       nan
Epoch  10/30  | train_loss =       nan | val_loss =       nan
Epoch  11/30  | train_loss =       nan | val_loss =       nan
Epoch  12/30  | train_loss =       nan | val_loss =       nan
Epoch  13/30  | train_loss =       nan | val_loss =       nan
Epoch  14/30  | train_loss =       nan | val_loss =       nan
Epoch  15/30  | train_loss =       nan | val_loss =       nan
Epoch  16/30  | train_loss =       nan | val_loss =       nan
Epoch  17/30  | train_loss =       nan | val_loss =       nan
Epoch  18/30  | train_loss =       nan | val_loss =       nan
Epoch  19/30  | train_loss =       nan | val_loss =       nan
Epoch  20/30  | train_loss =       nan | val_loss =       nan
Epoch  21/30  | train_loss =       nan | val_loss =       nan
Epoch  22/30  | train_loss =       nan | val_loss =       nan
Epoch  23/30  | train_loss =       nan | val_loss =       nan
```

```
Epoch  24/30  | train_loss =       nan | val_loss =       nan
Epoch  25/30  | train_loss =       nan | val_loss =       nan
Epoch  26/30  | train_loss =       nan | val_loss =       nan
Epoch  27/30  | train_loss =       nan | val_loss =       nan
Epoch  28/30  | train_loss =       nan | val_loss =       nan
Epoch  29/30  | train_loss =       nan | val_loss =       nan
Epoch  30/30  | train_loss =       nan | val_loss =       nan

Training model with lr = 10
Epoch   1/30  | train_loss =       nan | val_loss =       nan
Epoch   2/30  | train_loss =       nan | val_loss =       nan
Epoch   3/30  | train_loss =       nan | val_loss =       nan
Epoch   4/30  | train_loss =       nan | val_loss =       nan
Epoch   5/30  | train_loss =       nan | val_loss =       nan
Epoch   6/30  | train_loss =       nan | val_loss =       nan
Epoch   7/30  | train_loss =       nan | val_loss =       nan
Epoch   8/30  | train_loss =       nan | val_loss =       nan
Epoch   9/30  | train_loss =       nan | val_loss =       nan
Epoch  10/30  | train_loss =       nan | val_loss =       nan
Epoch  11/30  | train_loss =       nan | val_loss =       nan
Epoch  12/30  | train_loss =       nan | val_loss =       nan
Epoch  13/30  | train_loss =       nan | val_loss =       nan
Epoch  14/30  | train_loss =       nan | val_loss =       nan
Epoch  15/30  | train_loss =       nan | val_loss =       nan
Epoch  16/30  | train_loss =       nan | val_loss =       nan
Epoch  17/30  | train_loss =       nan | val_loss =       nan
Epoch  18/30  | train_loss =       nan | val_loss =       nan
Epoch  19/30  | train_loss =       nan | val_loss =       nan
Epoch  20/30  | train_loss =       nan | val_loss =       nan
Epoch  21/30  | train_loss =       nan | val_loss =       nan
Epoch  22/30  | train_loss =       nan | val_loss =       nan
Epoch  23/30  | train_loss =       nan | val_loss =       nan
Epoch  24/30  | train_loss =       nan | val_loss =       nan
Epoch  25/30  | train_loss =       nan | val_loss =       nan
Epoch  26/30  | train_loss =       nan | val_loss =       nan
Epoch  27/30  | train_loss =       nan | val_loss =       nan
Epoch  28/30  | train_loss =       nan | val_loss =       nan
Epoch  29/30  | train_loss =       nan | val_loss =       nan
Epoch  30/30  | train_loss =       nan | val_loss =       nan
```

[45]:
```
## TODO plot the losses from different models. What do you see and why?
loss_df = pd.DataFrame(all_losses).dropna(subset = ["mean_val_loss"]) # wirft␣
  ↪alle NaNs raus
loss_df["lr"] = loss_df["lr"].astype(str) # für schönere Farben

plt.figure(figsize = (8, 6))
```

```
sns.lineplot(
    data = loss_df,
    x = "epoch",
    y = "mean_val_loss",
    hue = "lr",
    marker = "o",
    linestyle = "--",
    palette = "tab10"
)

plt.title("Epoch vs. Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Validation Loss")

plt.xlim(0, loss_df["epoch"].max() + 0.5)

sns.despine()
plt.legend(title = "Learning rate")
plt.show()
```



6.1 How do we know the amount of epochs and best learning rates?

The honest answer - we just try it out. The heuristics, which are typically used are the following * use adaptive optimizers. Adam would be a typical example. It is an adaptive learning rate optimization algorithm that is designed to be appropriate for non-stationary objectives and problems with very noisy and/or sparse gradients. More details here. This makes the training more robust to the choise of the the learning rate * Use schedulers for the training. They change the value of the learning rate based on the loss behaviour. The most typical ones are * Early stopper . The early stopper is helpful to avoid redundunt computations and overfitting. It basically stops the train loop if the loss function does not decrease on the validation split for some time. * Warm up. Warm up slowly increases the learning rate in the begining of the training. This helps to avoid bad influence of not-so-good initialisation and especially helpful for the layers, which need to accumulate statistics, such as BatchNorm. We will use such layers later in the course.

6.1.1 Task

Modify the the training function with the early stopper logic. This should stop the training loop if the validation loss function does not improve over N epochs. The improvement is only something, which is better than the *tol* value, which stays for the tolerance.

Use $tol = 2$, $N = 5$, $num\_epochs = 150$, $lr = 0.01$ for the function start.

Question: * Can $tol = 0$? Why? * How many epochs it would really run? Try to change the $lr = 0.001$ and $lr = 0.0001$ and see when this would stop. * What happens if you increase the tolerance? * What if you increase N?

Yes, $tol = 0$ is possible, but it is often not ideal. Validation loss can fluctuate slightly due to stochastic training, so requiring any tiny improvement may make early stopping less stable.

Early stopping stops training once the validation loss no longer improves sufficiently. This avoids unnecessary training and can reduce overfitting. A larger tolerance makes stopping more strict, while a larger patience N allows the model to train longer even if the model doesnt improve over some epochs.

[46]:
```
## TODO: adopt the train function with the early stopper logic

def run_training_with_early_stopping(model, train_dataloader, val_dataloader,
                                     loss_function, optimizer, device,
                                     num_epochs = 150, tol = 2 , N = 5):
    losses = []
    best_val_loss = float("inf")
    no_improve_counter = 0

    for epoch in range(num_epochs):

        mean_train_loss = train_model(model, train_dataloader, loss_function,␣
 ↪optimizer, device)
        mean_val_loss = validate_model(model, val_dataloader, loss_function,␣
 ↪device)
```

```
        # Early stopping
        improvement = best_val_loss - mean_val_loss

        if improvement > tol:
            best_val_loss = mean_val_loss
            no_improve_counter = 0
        else:
            no_improve_counter += 1

        if no_improve_counter >= N:
            print(f"Early stopping at epoch {epoch+1}")
            break

        losses.append({"epoch": epoch + 1, "mean_train_loss": mean_train_loss,␣
↪"mean_val_loss": mean_val_loss})
        print(f"Epoch {epoch+1:3d}/{num_epochs:<3d} | train_loss =␣
↪{mean_train_loss:10.4f} | val_loss = {mean_val_loss:10.4f}")

    return losses
```

[47]:
```
## TODO: Train models with early stopping with the different learning rates

tol = 2
N = 5
num_epochs = 150

learning_rates = [0.0001, 0.001, 0.01, 0.1, 1, 10]

all_losses = []

for lr in learning_rates:

    print(f"\nTraining model with lr = {lr}")

    model = LinearRegression(input_dim = input_dim, output_dim = output_dim).
↪float()
    model = model.to(device)

    loss_function = torch.nn.MSELoss()
    optimizer = torch.optim.SGD(model.parameters(), lr = lr)

    losses = run_training_with_early_stopping(model, train_dataloader,␣
↪val_dataloader, loss_function, optimizer,
                                              device, num_epochs, tol, N)

    for loss in losses:
        loss["lr"] = lr
```

```
        all_losses.append(loss)
```

```
Training model with lr = 0.0001
Epoch   1/150 | train_loss =  2184.4193 | val_loss =  2142.1808
Epoch   2/150 | train_loss =  1946.2795 | val_loss =  1948.7608
Epoch   3/150 | train_loss =  1811.1248 | val_loss =  1828.5277
Epoch   4/150 | train_loss =  1727.0954 | val_loss =  1759.2832
Epoch   5/150 | train_loss =  1668.4921 | val_loss =  1693.7270
Epoch   6/150 | train_loss =  1621.7882 | val_loss =  1657.7829
Epoch   7/150 | train_loss =  1582.5820 | val_loss =  1614.6998
Epoch   8/150 | train_loss =  1545.7602 | val_loss =  1591.8373
Epoch   9/150 | train_loss =  1513.5256 | val_loss =  1549.0102
Epoch  10/150 | train_loss =  1482.7570 | val_loss =  1523.3578
Epoch  11/150 | train_loss =  1453.8044 | val_loss =  1501.2830
Epoch  12/150 | train_loss =  1425.0958 | val_loss =  1465.9087
Epoch  13/150 | train_loss =  1397.7871 | val_loss =  1453.9990
Epoch  14/150 | train_loss =  1372.8871 | val_loss =  1421.5653
Epoch  15/150 | train_loss =  1348.4961 | val_loss =  1403.4206
Epoch  16/150 | train_loss =  1324.0400 | val_loss =  1378.1387
Epoch  17/150 | train_loss =  1301.0500 | val_loss =  1343.7932
Epoch  18/150 | train_loss =  1279.1439 | val_loss =  1327.6279
Epoch  19/150 | train_loss =  1258.3859 | val_loss =  1298.8403
Epoch  20/150 | train_loss =  1236.8783 | val_loss =  1276.0654
Epoch  21/150 | train_loss =  1216.7687 | val_loss =  1262.8984
Epoch  22/150 | train_loss =  1197.2002 | val_loss =  1249.7485
Epoch  23/150 | train_loss =  1178.2935 | val_loss =  1234.5036
Epoch  24/150 | train_loss =  1158.9402 | val_loss =  1201.1870
Epoch  25/150 | train_loss =  1141.0922 | val_loss =  1191.7386
Epoch  26/150 | train_loss =  1123.1861 | val_loss =  1170.0030
Epoch  27/150 | train_loss =  1106.7715 | val_loss =  1150.6846
Epoch  28/150 | train_loss =  1089.9075 | val_loss =  1132.8732
Epoch  29/150 | train_loss =  1073.7562 | val_loss =  1115.0883
Epoch  30/150 | train_loss =  1057.8631 | val_loss =  1101.5120
Epoch  31/150 | train_loss =  1043.0892 | val_loss =  1089.0932
Epoch  32/150 | train_loss =  1027.8647 | val_loss =  1074.7886
Epoch  33/150 | train_loss =  1013.3400 | val_loss =  1065.4477
Epoch  34/150 | train_loss =   999.4183 | val_loss =  1043.2528
Epoch  35/150 | train_loss =   985.1375 | val_loss =  1035.8852
Epoch  36/150 | train_loss =   971.7588 | val_loss =  1024.8604
Epoch  37/150 | train_loss =   958.0908 | val_loss =  1007.3277
Epoch  38/150 | train_loss =   945.7302 | val_loss =   996.1717
Epoch  39/150 | train_loss =   933.1930 | val_loss =   975.3278
Epoch  40/150 | train_loss =   921.5751 | val_loss =   967.2066
Epoch  41/150 | train_loss =   909.0228 | val_loss =   956.7079
Epoch  42/150 | train_loss =   897.6593 | val_loss =   945.2851
Epoch  43/150 | train_loss =   886.4478 | val_loss =   937.4911
Epoch  44/150 | train_loss =   875.0661 | val_loss =   921.3555
```

```
Epoch  45/150 | train_loss =   864.4270 | val_loss =   907.5549
Epoch  46/150 | train_loss =   853.6877 | val_loss =   900.3962
Epoch  47/150 | train_loss =   843.3543 | val_loss =   894.7594
Epoch  48/150 | train_loss =   833.5131 | val_loss =   873.9525
Epoch  49/150 | train_loss =   823.7813 | val_loss =   867.1814
Epoch  50/150 | train_loss =   813.6003 | val_loss =   859.6815
Epoch  51/150 | train_loss =   804.8343 | val_loss =   846.5221
Epoch  52/150 | train_loss =   795.2132 | val_loss =   837.9453
Epoch  53/150 | train_loss =   787.0957 | val_loss =   832.7656
Epoch  54/150 | train_loss =   777.9165 | val_loss =   821.0149
Epoch  55/150 | train_loss =   769.7006 | val_loss =   811.0271
Epoch  56/150 | train_loss =   760.9302 | val_loss =   802.2051
Epoch  57/150 | train_loss =   752.7492 | val_loss =   793.5519
Epoch  58/150 | train_loss =   745.0323 | val_loss =   788.3548
Epoch  59/150 | train_loss =   737.4255 | val_loss =   775.3740
Epoch  60/150 | train_loss =   729.7796 | val_loss =   777.3241
Epoch  61/150 | train_loss =   722.6072 | val_loss =   766.7503
Epoch  62/150 | train_loss =   714.9687 | val_loss =   759.3920
Epoch  63/150 | train_loss =   707.8971 | val_loss =   751.7610
Epoch  64/150 | train_loss =   701.0894 | val_loss =   742.9882
Epoch  65/150 | train_loss =   694.4541 | val_loss =   740.8741
Epoch  66/150 | train_loss =   687.6780 | val_loss =   730.3001
Epoch  67/150 | train_loss =   681.3059 | val_loss =   724.8989
Epoch  68/150 | train_loss =   675.1526 | val_loss =   710.3872
Epoch  69/150 | train_loss =   668.9282 | val_loss =   708.9215
Epoch  70/150 | train_loss =   663.3405 | val_loss =   702.0129
Epoch  71/150 | train_loss =   657.1962 | val_loss =   703.5068
Epoch  72/150 | train_loss =   651.3185 | val_loss =   690.0810
Epoch  73/150 | train_loss =   645.7661 | val_loss =   684.3768
Epoch  74/150 | train_loss =   640.3053 | val_loss =   679.7404
Epoch  75/150 | train_loss =   634.9275 | val_loss =   685.7345
Epoch  76/150 | train_loss =   629.8088 | val_loss =   671.8145
Epoch  77/150 | train_loss =   624.8620 | val_loss =   664.9487
Epoch  78/150 | train_loss =   619.8032 | val_loss =   653.3274
Epoch  79/150 | train_loss =   614.5115 | val_loss =   661.0618
Epoch  80/150 | train_loss =   609.9887 | val_loss =   647.8994
Epoch  81/150 | train_loss =   605.1840 | val_loss =   642.0400
Epoch  82/150 | train_loss =   600.7364 | val_loss =   639.7286
Epoch  83/150 | train_loss =   596.2687 | val_loss =   637.5082
Epoch  84/150 | train_loss =   592.0496 | val_loss =   638.3421
Epoch  85/150 | train_loss =   587.8655 | val_loss =   627.8058
Epoch  86/150 | train_loss =   583.8572 | val_loss =   622.1388
Epoch  87/150 | train_loss =   579.7885 | val_loss =   621.8780
Epoch  88/150 | train_loss =   575.9617 | val_loss =   610.4195
Epoch  89/150 | train_loss =   571.4467 | val_loss =   611.3204
Epoch  90/150 | train_loss =   568.0242 | val_loss =   614.0332
Epoch  91/150 | train_loss =   563.9886 | val_loss =   598.0083
Epoch  92/150 | train_loss =   560.2228 | val_loss =   598.3803
```

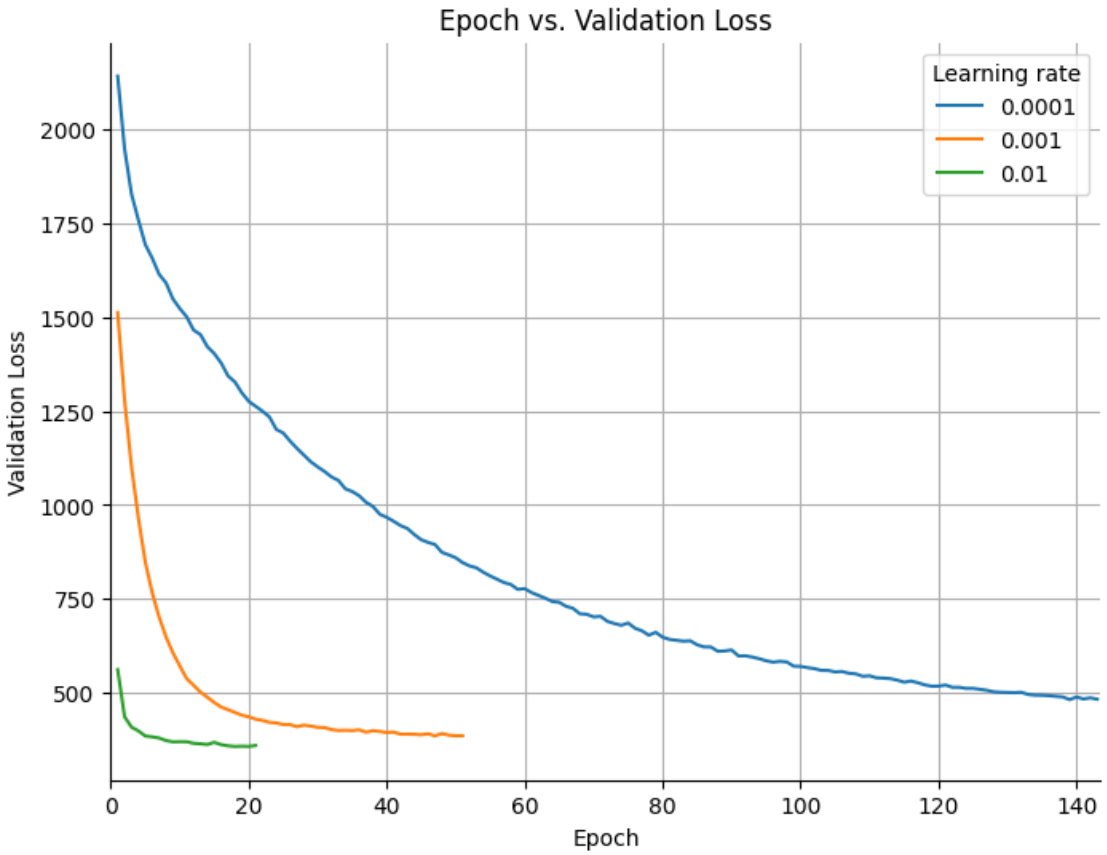
```
Epoch  93/150 | train_loss =   556.8299 | val_loss =   595.1137
Epoch  94/150 | train_loss =   553.4482 | val_loss =   590.5911
Epoch  95/150 | train_loss =   549.9423 | val_loss =   585.3567
Epoch  96/150 | train_loss =   546.4145 | val_loss =   581.4267
Epoch  97/150 | train_loss =   543.7795 | val_loss =   583.6920
Epoch  98/150 | train_loss =   540.3427 | val_loss =   581.9537
Epoch  99/150 | train_loss =   537.2088 | val_loss =   570.8360
Epoch 100/150 | train_loss =   534.1599 | val_loss =   570.1757
Epoch 101/150 | train_loss =   531.0732 | val_loss =   566.8960
Epoch 102/150 | train_loss =   528.1997 | val_loss =   564.3508
Epoch 103/150 | train_loss =   525.2894 | val_loss =   559.6760
Epoch 104/150 | train_loss =   522.8924 | val_loss =   559.4040
Epoch 105/150 | train_loss =   519.7843 | val_loss =   555.6218
Epoch 106/150 | train_loss =   516.9736 | val_loss =   556.6364
Epoch 107/150 | train_loss =   514.2432 | val_loss =   551.9746
Epoch 108/150 | train_loss =   511.9609 | val_loss =   550.3620
Epoch 109/150 | train_loss =   509.6078 | val_loss =   543.9608
Epoch 110/150 | train_loss =   507.4811 | val_loss =   545.1969
Epoch 111/150 | train_loss =   504.3984 | val_loss =   539.7756
Epoch 112/150 | train_loss =   502.3847 | val_loss =   538.9624
Epoch 113/150 | train_loss =   500.1588 | val_loss =   537.6104
Epoch 114/150 | train_loss =   497.9257 | val_loss =   533.7562
Epoch 115/150 | train_loss =   495.2484 | val_loss =   528.1803
Epoch 116/150 | train_loss =   493.3120 | val_loss =   531.1776
Epoch 117/150 | train_loss =   491.4089 | val_loss =   526.7036
Epoch 118/150 | train_loss =   489.3054 | val_loss =   520.9800
Epoch 119/150 | train_loss =   487.2843 | val_loss =   517.4182
Epoch 120/150 | train_loss =   485.1076 | val_loss =   517.6432
Epoch 121/150 | train_loss =   483.2783 | val_loss =   520.6100
Epoch 122/150 | train_loss =   481.3840 | val_loss =   514.4901
Epoch 123/150 | train_loss =   479.5567 | val_loss =   514.5144
Epoch 124/150 | train_loss =   477.6370 | val_loss =   511.6244
Epoch 125/150 | train_loss =   475.9661 | val_loss =   511.7337
Epoch 126/150 | train_loss =   474.0837 | val_loss =   509.1695
Epoch 127/150 | train_loss =   472.5374 | val_loss =   506.5556
Epoch 128/150 | train_loss =   470.5462 | val_loss =   502.5280
Epoch 129/150 | train_loss =   469.3877 | val_loss =   501.3308
Epoch 130/150 | train_loss =   467.9489 | val_loss =   500.7240
Epoch 131/150 | train_loss =   465.8918 | val_loss =   500.3285
Epoch 132/150 | train_loss =   464.2877 | val_loss =   501.2717
Epoch 133/150 | train_loss =   462.9525 | val_loss =   495.4287
Epoch 134/150 | train_loss =   461.3021 | val_loss =   493.5841
Epoch 135/150 | train_loss =   460.0033 | val_loss =   493.3346
Epoch 136/150 | train_loss =   458.1876 | val_loss =   492.2034
Epoch 137/150 | train_loss =   457.3474 | val_loss =   490.5035
Epoch 138/150 | train_loss =   455.8133 | val_loss =   488.9072
Epoch 139/150 | train_loss =   454.5778 | val_loss =   481.6821
Epoch 140/150 | train_loss =   453.2730 | val_loss =   488.8089
```

```
Epoch 141/150 | train_loss =   451.8755 | val_loss =   483.4528
Epoch 142/150 | train_loss =   450.7399 | val_loss =   486.5043
Epoch 143/150 | train_loss =   449.3795 | val_loss =   482.8091
Early stopping at epoch 144

Training model with lr = 0.001
Epoch   1/150 | train_loss =  1709.8973 | val_loss =  1512.6647
Epoch   2/150 | train_loss =  1340.5256 | val_loss =  1279.0085
Epoch   3/150 | train_loss =  1133.8128 | val_loss =  1102.5616
Epoch   4/150 | train_loss =   980.5530 | val_loss =   965.6708
Epoch   5/150 | train_loss =   859.7248 | val_loss =   848.3833
Epoch   6/150 | train_loss =   765.9483 | val_loss =   766.7526
Epoch   7/150 | train_loss =   693.1115 | val_loss =   701.9247
Epoch   8/150 | train_loss =   632.8948 | val_loss =   649.0917
Epoch   9/150 | train_loss =   585.7481 | val_loss =   606.6732
Epoch  10/150 | train_loss =   548.4438 | val_loss =   571.6676
Epoch  11/150 | train_loss =   518.6098 | val_loss =   537.6268
Epoch  12/150 | train_loss =   494.1357 | val_loss =   520.0017
Epoch  13/150 | train_loss =   475.0986 | val_loss =   501.7604
Epoch  14/150 | train_loss =   458.9948 | val_loss =   488.0975
Epoch  15/150 | train_loss =   446.4264 | val_loss =   474.0732
Epoch  16/150 | train_loss =   436.0560 | val_loss =   461.9172
Epoch  17/150 | train_loss =   427.4426 | val_loss =   454.9657
Epoch  18/150 | train_loss =   419.7637 | val_loss =   447.5961
Epoch  19/150 | train_loss =   414.0281 | val_loss =   440.2106
Epoch  20/150 | train_loss =   408.8701 | val_loss =   435.9410
Epoch  21/150 | train_loss =   404.4622 | val_loss =   429.6660
Epoch  22/150 | train_loss =   400.6685 | val_loss =   426.3889
Epoch  23/150 | train_loss =   397.8370 | val_loss =   421.2135
Epoch  24/150 | train_loss =   394.7447 | val_loss =   419.6587
Epoch  25/150 | train_loss =   392.1046 | val_loss =   415.4460
Epoch  26/150 | train_loss =   389.9233 | val_loss =   415.1065
Epoch  27/150 | train_loss =   387.9640 | val_loss =   409.6300
Epoch  28/150 | train_loss =   386.1714 | val_loss =   413.7491
Epoch  29/150 | train_loss =   384.3427 | val_loss =   411.2545
Epoch  30/150 | train_loss =   382.6704 | val_loss =   407.9706
Epoch  31/150 | train_loss =   381.4769 | val_loss =   407.2902
Epoch  32/150 | train_loss =   380.1789 | val_loss =   401.9559
Epoch  33/150 | train_loss =   378.8641 | val_loss =   399.4582
Epoch  34/150 | train_loss =   377.6448 | val_loss =   400.0814
Epoch  35/150 | train_loss =   376.5295 | val_loss =   399.2744
Epoch  36/150 | train_loss =   375.5033 | val_loss =   401.1769
Epoch  37/150 | train_loss =   374.7498 | val_loss =   395.0212
Epoch  38/150 | train_loss =   373.5852 | val_loss =   399.0864
Epoch  39/150 | train_loss =   372.7197 | val_loss =   396.9109
Epoch  40/150 | train_loss =   372.1370 | val_loss =   394.1527
Epoch  41/150 | train_loss =   371.0776 | val_loss =   395.1344
Epoch  42/150 | train_loss =   370.3644 | val_loss =   389.7717
```

```
Epoch  43/150 | train_loss =   369.7421 | val_loss =   389.7492
Epoch  44/150 | train_loss =   368.9746 | val_loss =   389.4257
Epoch  45/150 | train_loss =   368.3412 | val_loss =   388.1946
Epoch  46/150 | train_loss =   367.4248 | val_loss =   390.6768
Epoch  47/150 | train_loss =   366.9017 | val_loss =   385.3249
Epoch  48/150 | train_loss =   366.2947 | val_loss =   391.3650
Epoch  49/150 | train_loss =   365.9867 | val_loss =   387.1354
Epoch  50/150 | train_loss =   365.2305 | val_loss =   385.4508
Epoch  51/150 | train_loss =   364.5408 | val_loss =   385.6552
Early stopping at epoch 52

Training model with lr = 0.01
Epoch   1/150 | train_loss =   938.9085 | val_loss =   561.7311
Epoch   2/150 | train_loss =   451.1852 | val_loss =   436.0847
Epoch   3/150 | train_loss =   393.3018 | val_loss =   408.4870
Epoch   4/150 | train_loss =   377.3736 | val_loss =   398.4570
Epoch   5/150 | train_loss =   369.4594 | val_loss =   384.9360
Epoch   6/150 | train_loss =   363.7288 | val_loss =   382.5487
Epoch   7/150 | train_loss =   360.3532 | val_loss =   379.4608
Epoch   8/150 | train_loss =   356.6361 | val_loss =   372.8480
Epoch   9/150 | train_loss =   354.2417 | val_loss =   369.2874
Epoch  10/150 | train_loss =   352.3906 | val_loss =   369.6928
Epoch  11/150 | train_loss =   350.7379 | val_loss =   369.6007
Epoch  12/150 | train_loss =   349.7548 | val_loss =   365.3718
Epoch  13/150 | train_loss =   348.6143 | val_loss =   364.0321
Epoch  14/150 | train_loss =   347.3761 | val_loss =   362.2629
Epoch  15/150 | train_loss =   346.0851 | val_loss =   367.8715
Epoch  16/150 | train_loss =   345.5193 | val_loss =   361.8503
Epoch  17/150 | train_loss =   344.5219 | val_loss =   358.6938
Epoch  18/150 | train_loss =   343.9682 | val_loss =   357.0193
Epoch  19/150 | train_loss =   342.9235 | val_loss =   357.7003
Epoch  20/150 | train_loss =   342.1884 | val_loss =   357.0609
Epoch  21/150 | train_loss =   341.6271 | val_loss =   359.5107
Early stopping at epoch 22

Training model with lr = 0.1
Epoch   1/150 | train_loss =       nan | val_loss =       nan
Epoch   2/150 | train_loss =       nan | val_loss =       nan
Epoch   3/150 | train_loss =       nan | val_loss =       nan
Epoch   4/150 | train_loss =       nan | val_loss =       nan
Early stopping at epoch 5

Training model with lr = 1
Epoch   1/150 | train_loss =       nan | val_loss =       nan
Epoch   2/150 | train_loss =       nan | val_loss =       nan
Epoch   3/150 | train_loss =       nan | val_loss =       nan
Epoch   4/150 | train_loss =       nan | val_loss =       nan
Early stopping at epoch 5
```

```
Training model with lr = 10
Epoch   1/150 | train_loss =        nan | val_loss =        nan
Epoch   2/150 | train_loss =        nan | val_loss =        nan
Epoch   3/150 | train_loss =        nan | val_loss =        nan
Epoch   4/150 | train_loss =        nan | val_loss =        nan
Early stopping at epoch 5
```

[48]:
```python
## TODO: Visualize the results. What do you see and why?
loss_df = pd.DataFrame(all_losses).dropna(subset = ["mean_val_loss"]) # wirft␣
 ↪alle NaNs raus
loss_df["lr"] = loss_df["lr"].astype(str) # für schönere Farben

plt.figure(figsize = (8, 6))

sns.lineplot(
    data = loss_df,
    x = "epoch",
    y = "mean_val_loss",
    hue = "lr",
    palette = "tab10"
)

plt.title("Epoch vs. Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Validation Loss")

plt.xlim(0, loss_df["epoch"].max() + 0.5)

sns.despine()
plt.legend(title = "Learning rate")
plt.grid()
plt.show()
```



6.2 Now lets try to explore the impact of the batch size

Task: Fix $num\_epochs = 150$ and $lr = 0.001$ and try to change the batch size using values $[2, 4, 32, 128, 1024]$ for the batch size. * What impact the batch size have? Why?

Batch size affects the stability and speed of learning. Small batches produce noisier but frequent updates and can sometimes generalize better. Larger batches give smoother loss curves, but they update the model less often per epoch and may learn more slowly.

[49]:
```
num_epochs = 150
lr = 0.001
## TODO: Train with different batch sizes

batch_sizes = [4, 32, 128, 1024]

all_losses = []

for bs in batch_sizes:

    print(f"\nTraining model with batch size = {bs}")
```

```
    train_dataloader = DataLoader(conductor_train, batch_size = bs, shuffle =␣
↪True)
    val_dataloader = DataLoader(conductor_val, batch_size = bs, shuffle = False)

    model = LinearRegression(input_dim = input_dim, output_dim = output_dim).
↪float()
    model = model.to(device)

    loss_function = torch.nn.MSELoss()
    optimizer = torch.optim.SGD(model.parameters(), lr = lr)

    losses = run_training_with_early_stopping(model, train_dataloader,␣
↪val_dataloader, loss_function, optimizer,
                                              device, num_epochs, )

    for loss in losses:
        loss["batch_size"] = bs
        all_losses.append(loss)
```

```
Training model with batch size = 4
Epoch   1/150 | train_loss =   480.4268 | val_loss =   387.2101
Epoch   2/150 | train_loss =   357.4866 | val_loss =   367.1894
Epoch   3/150 | train_loss =   350.3832 | val_loss =   365.2566
Epoch   4/150 | train_loss =   344.9020 | val_loss =   359.8296
Epoch   5/150 | train_loss =   341.4002 | val_loss =   352.9915
Epoch   6/150 | train_loss =   339.5708 | val_loss =   349.0490
Epoch   7/150 | train_loss =   336.8982 | val_loss =   347.1563
Epoch   8/150 | train_loss =   335.0543 | val_loss =   347.7784
Epoch   9/150 | train_loss =   333.1638 | val_loss =   344.4590
Epoch  10/150 | train_loss =   331.5732 | val_loss =   349.2240
Epoch  11/150 | train_loss =   330.3427 | val_loss =   349.3535
Epoch  12/150 | train_loss =   329.1659 | val_loss =   340.2989
Epoch  13/150 | train_loss =   328.4785 | val_loss =   342.6094
Epoch  14/150 | train_loss =   327.2018 | val_loss =   342.2946
Epoch  15/150 | train_loss =   326.9641 | val_loss =   345.8200
Epoch  16/150 | train_loss =   326.1503 | val_loss =   338.1694
Epoch  17/150 | train_loss =   325.6671 | val_loss =   348.6403
Epoch  18/150 | train_loss =   324.4858 | val_loss =   336.0838
Epoch  19/150 | train_loss =   324.1182 | val_loss =   336.9103
Epoch  20/150 | train_loss =   323.6330 | val_loss =   340.1175
Epoch  21/150 | train_loss =   322.7666 | val_loss =   337.2104
Epoch  22/150 | train_loss =   321.8798 | val_loss =   340.3215
Early stopping at epoch 23

Training model with batch size = 32
```

```
Epoch   1/150 | train_loss =  1013.1831 | val_loss =   641.7104
Epoch   2/150 | train_loss =   494.8941 | val_loss =   457.7122
Epoch   3/150 | train_loss =   408.3751 | val_loss =   417.4288
Epoch   4/150 | train_loss =   386.2383 | val_loss =   401.9775
Epoch   5/150 | train_loss =   376.1049 | val_loss =   394.4634
Epoch   6/150 | train_loss =   369.5416 | val_loss =   388.6962
Epoch   7/150 | train_loss =   365.0571 | val_loss =   383.8458
Epoch   8/150 | train_loss =   361.0819 | val_loss =   381.3210
Epoch   9/150 | train_loss =   358.4649 | val_loss =   378.6006
Epoch  10/150 | train_loss =   356.4221 | val_loss =   376.3549
Epoch  11/150 | train_loss =   354.2987 | val_loss =   373.4864
Epoch  12/150 | train_loss =   352.8486 | val_loss =   373.5231
Epoch  13/150 | train_loss =   351.7707 | val_loss =   372.0538
Epoch  14/150 | train_loss =   350.2720 | val_loss =   370.4236
Epoch  15/150 | train_loss =   349.4783 | val_loss =   369.3405
Epoch  16/150 | train_loss =   348.3447 | val_loss =   367.3338
Epoch  17/150 | train_loss =   347.8284 | val_loss =   366.2974
Epoch  18/150 | train_loss =   346.6259 | val_loss =   366.7164
Epoch  19/150 | train_loss =   345.7756 | val_loss =   364.8937
Epoch  20/150 | train_loss =   345.0966 | val_loss =   364.4394
Epoch  21/150 | train_loss =   344.3254 | val_loss =   363.7436
Epoch  22/150 | train_loss =   343.7296 | val_loss =   362.9963
Epoch  23/150 | train_loss =   343.2066 | val_loss =   361.7272
Epoch  24/150 | train_loss =   342.4668 | val_loss =   360.7169
Epoch  25/150 | train_loss =   341.9167 | val_loss =   360.9133
Epoch  26/150 | train_loss =   342.0598 | val_loss =   360.4435
Epoch  27/150 | train_loss =   340.9316 | val_loss =   359.5320
Epoch  28/150 | train_loss =   340.6963 | val_loss =   359.5175
Epoch  29/150 | train_loss =   340.7910 | val_loss =   359.8683
Epoch  30/150 | train_loss =   339.9175 | val_loss =   358.1719
Epoch  31/150 | train_loss =   339.1405 | val_loss =   357.7459
Early stopping at epoch 32

Training model with batch size = 128
Epoch   1/150 | train_loss =  1525.6459 | val_loss =  1255.2296
Epoch   2/150 | train_loss =  1054.9264 | val_loss =   946.6952
Epoch   3/150 | train_loss =   812.9475 | val_loss =   758.3447
Epoch   4/150 | train_loss =   661.4996 | val_loss =   640.1166
Epoch   5/150 | train_loss =   567.4932 | val_loss =   564.7068
Epoch   6/150 | train_loss =   506.6358 | val_loss =   516.5652
Epoch   7/150 | train_loss =   467.2102 | val_loss =   485.2468
Epoch   8/150 | train_loss =   441.3177 | val_loss =   464.0526
Epoch   9/150 | train_loss =   423.8667 | val_loss =   449.7628
Epoch  10/150 | train_loss =   411.9966 | val_loss =   439.2545
Epoch  11/150 | train_loss =   402.6879 | val_loss =   431.4860
Epoch  12/150 | train_loss =   396.6220 | val_loss =   425.8883
Epoch  13/150 | train_loss =   391.0233 | val_loss =   421.4508
Epoch  14/150 | train_loss =   386.8416 | val_loss =   417.4396
```

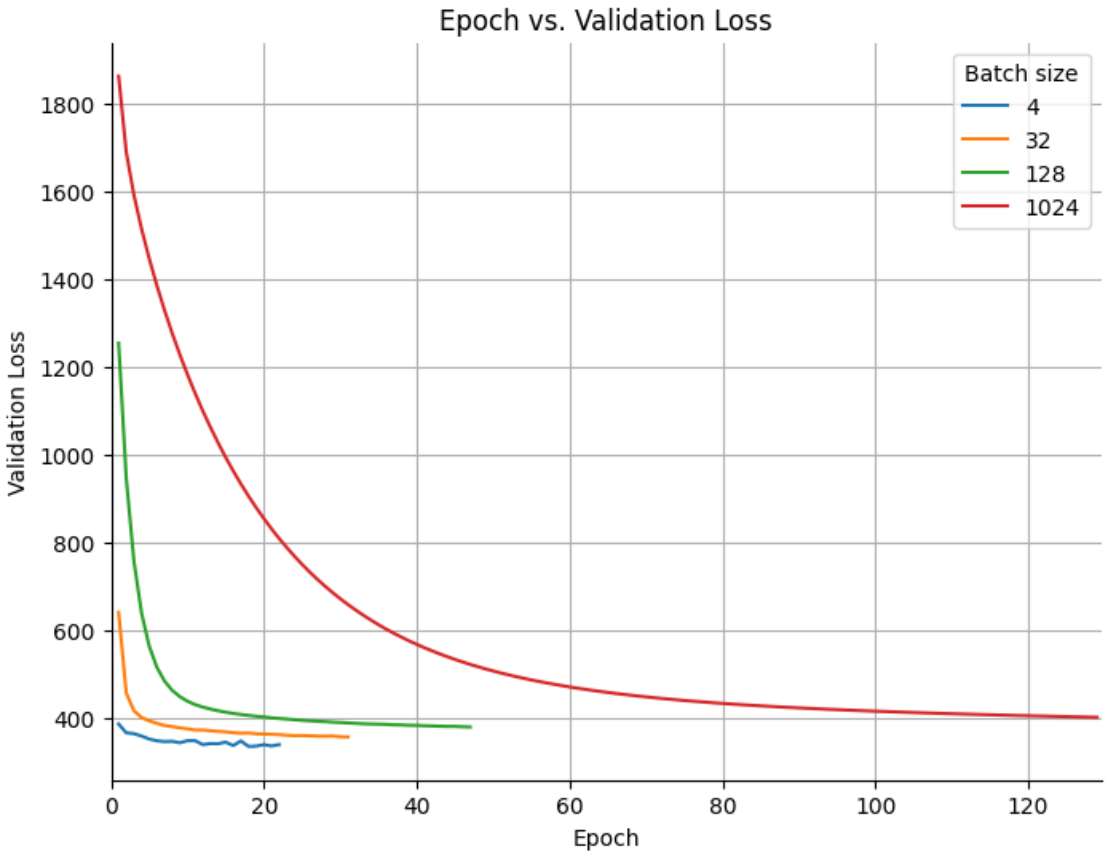
```
Epoch  15/150 | train_loss =   383.5643 | val_loss =   414.2733
Epoch  16/150 | train_loss =   380.8760 | val_loss =   411.2901
Epoch  17/150 | train_loss =   378.5571 | val_loss =   408.9909
Epoch  18/150 | train_loss =   375.9953 | val_loss =   406.9550
Epoch  19/150 | train_loss =   374.6222 | val_loss =   404.7890
Epoch  20/150 | train_loss =   372.1180 | val_loss =   403.4598
Epoch  21/150 | train_loss =   371.0513 | val_loss =   401.1913
Epoch  22/150 | train_loss =   369.4365 | val_loss =   399.8781
Epoch  23/150 | train_loss =   368.0755 | val_loss =   398.4870
Epoch  24/150 | train_loss =   366.8771 | val_loss =   397.1199
Epoch  25/150 | train_loss =   365.7626 | val_loss =   395.6822
Epoch  26/150 | train_loss =   364.5924 | val_loss =   394.4586
Epoch  27/150 | train_loss =   363.7689 | val_loss =   393.4406
Epoch  28/150 | train_loss =   362.3980 | val_loss =   392.6740
Epoch  29/150 | train_loss =   361.4757 | val_loss =   391.3851
Epoch  30/150 | train_loss =   361.0851 | val_loss =   390.6880
Epoch  31/150 | train_loss =   359.9318 | val_loss =   389.6349
Epoch  32/150 | train_loss =   359.3171 | val_loss =   388.7828
Epoch  33/150 | train_loss =   358.6077 | val_loss =   387.8557
Epoch  34/150 | train_loss =   358.0716 | val_loss =   387.2433
Epoch  35/150 | train_loss =   357.0635 | val_loss =   386.9327
Epoch  36/150 | train_loss =   356.8720 | val_loss =   386.3789
Epoch  37/150 | train_loss =   356.2137 | val_loss =   385.4188
Epoch  38/150 | train_loss =   356.0857 | val_loss =   385.2114
Epoch  39/150 | train_loss =   355.2685 | val_loss =   384.3167
Epoch  40/150 | train_loss =   354.5902 | val_loss =   384.0816
Epoch  41/150 | train_loss =   354.4393 | val_loss =   383.2782
Epoch  42/150 | train_loss =   353.7870 | val_loss =   382.7696
Epoch  43/150 | train_loss =   353.3309 | val_loss =   382.1909
Epoch  44/150 | train_loss =   352.8702 | val_loss =   381.8502
Epoch  45/150 | train_loss =   352.6457 | val_loss =   381.7212
Epoch  46/150 | train_loss =   352.4265 | val_loss =   380.9740
Epoch  47/150 | train_loss =   352.0431 | val_loss =   380.3955
Early stopping at epoch 48

Training model with batch size = 1024
Epoch   1/150 | train_loss =  2038.5413 | val_loss =  1863.8356
Epoch   2/150 | train_loss =  1716.1647 | val_loss =  1690.3190
Epoch   3/150 | train_loss =  1593.2855 | val_loss =  1592.7416
Epoch   4/150 | train_loss =  1507.5061 | val_loss =  1515.6566
Epoch   5/150 | train_loss =  1434.4118 | val_loss =  1448.1608
Epoch   6/150 | train_loss =  1368.9936 | val_loss =  1386.6960
Epoch   7/150 | train_loss =  1309.4102 | val_loss =  1330.3587
Epoch   8/150 | train_loss =  1254.3580 | val_loss =  1278.1679
Epoch   9/150 | train_loss =  1203.6029 | val_loss =  1229.4047
Epoch  10/150 | train_loss =  1156.5282 | val_loss =  1184.1048
Epoch  11/150 | train_loss =  1112.7687 | val_loss =  1141.3013
Epoch  12/150 | train_loss =  1071.6608 | val_loss =  1101.2195
```

```
Epoch  13/150 | train_loss =  1033.1460 | val_loss =  1063.8301
Epoch  14/150 | train_loss =   997.0005 | val_loss =  1028.5702
Epoch  15/150 | train_loss =   963.6379 | val_loss =   995.1066
Epoch  16/150 | train_loss =   931.5773 | val_loss =   963.8014
Epoch  17/150 | train_loss =   901.6039 | val_loss =   934.2743
Epoch  18/150 | train_loss =   873.5831 | val_loss =   906.4799
Epoch  19/150 | train_loss =   847.0377 | val_loss =   880.2474
Epoch  20/150 | train_loss =   822.1716 | val_loss =   855.5881
Epoch  21/150 | train_loss =   798.9152 | val_loss =   832.1318
Epoch  22/150 | train_loss =   776.8143 | val_loss =   810.0661
Epoch  23/150 | train_loss =   755.6584 | val_loss =   789.3464
Epoch  24/150 | train_loss =   736.2099 | val_loss =   769.6838
Epoch  25/150 | train_loss =   717.7534 | val_loss =   750.9884
Epoch  26/150 | train_loss =   700.0096 | val_loss =   733.5530
Epoch  27/150 | train_loss =   683.4950 | val_loss =   717.0045
Epoch  28/150 | train_loss =   668.1530 | val_loss =   701.3835
Epoch  29/150 | train_loss =   653.3413 | val_loss =   686.6093
Epoch  30/150 | train_loss =   639.3888 | val_loss =   672.6956
Epoch  31/150 | train_loss =   626.3848 | val_loss =   659.5691
Epoch  32/150 | train_loss =   613.8623 | val_loss =   647.1379
Epoch  33/150 | train_loss =   602.2442 | val_loss =   635.3459
Epoch  34/150 | train_loss =   591.3628 | val_loss =   624.1748
Epoch  35/150 | train_loss =   581.0217 | val_loss =   613.5391
Epoch  36/150 | train_loss =   571.0080 | val_loss =   603.5453
Epoch  37/150 | train_loss =   561.6314 | val_loss =   594.0519
Epoch  38/150 | train_loss =   552.9488 | val_loss =   585.1170
Epoch  39/150 | train_loss =   544.6969 | val_loss =   576.6343
Epoch  40/150 | train_loss =   536.6913 | val_loss =   568.6348
Epoch  41/150 | train_loss =   529.0447 | val_loss =   561.0257
Epoch  42/150 | train_loss =   522.0229 | val_loss =   553.8500
Epoch  43/150 | train_loss =   515.3622 | val_loss =   547.0129
Epoch  44/150 | train_loss =   509.1272 | val_loss =   540.5389
Epoch  45/150 | train_loss =   503.0938 | val_loss =   534.3784
Epoch  46/150 | train_loss =   497.5586 | val_loss =   528.5638
Epoch  47/150 | train_loss =   492.0525 | val_loss =   523.0174
Epoch  48/150 | train_loss =   486.9025 | val_loss =   517.7618
Epoch  49/150 | train_loss =   482.0523 | val_loss =   512.7684
Epoch  50/150 | train_loss =   477.4934 | val_loss =   508.0247
Epoch  51/150 | train_loss =   473.1464 | val_loss =   503.5287
Epoch  52/150 | train_loss =   468.7745 | val_loss =   499.2641
Epoch  53/150 | train_loss =   465.0632 | val_loss =   495.2065
Epoch  54/150 | train_loss =   461.3448 | val_loss =   491.3206
Epoch  55/150 | train_loss =   457.8039 | val_loss =   487.6172
Epoch  56/150 | train_loss =   454.4752 | val_loss =   484.1342
Epoch  57/150 | train_loss =   451.1395 | val_loss =   480.8246
Epoch  58/150 | train_loss =   448.1904 | val_loss =   477.6438
Epoch  59/150 | train_loss =   445.3968 | val_loss =   474.5877
Epoch  60/150 | train_loss =   442.4923 | val_loss =   471.6705
```

```
Epoch  61/150 | train_loss =   439.8130 | val_loss =   468.9015
Epoch  62/150 | train_loss =   437.2703 | val_loss =   466.2601
Epoch  63/150 | train_loss =   434.9713 | val_loss =   463.7576
Epoch  64/150 | train_loss =   432.7209 | val_loss =   461.3791
Epoch  65/150 | train_loss =   430.4307 | val_loss =   459.1125
Epoch  66/150 | train_loss =   428.3626 | val_loss =   456.9147
Epoch  67/150 | train_loss =   426.4306 | val_loss =   454.8291
Epoch  68/150 | train_loss =   424.5559 | val_loss =   452.8213
Epoch  69/150 | train_loss =   422.9049 | val_loss =   450.8745
Epoch  70/150 | train_loss =   420.9891 | val_loss =   449.0043
Epoch  71/150 | train_loss =   419.2935 | val_loss =   447.2594
Epoch  72/150 | train_loss =   417.8625 | val_loss =   445.5435
Epoch  73/150 | train_loss =   416.3067 | val_loss =   443.8982
Epoch  74/150 | train_loss =   414.9613 | val_loss =   442.3365
Epoch  75/150 | train_loss =   413.3084 | val_loss =   440.8269
Epoch  76/150 | train_loss =   412.1802 | val_loss =   439.3873
Epoch  77/150 | train_loss =   410.7146 | val_loss =   438.0055
Epoch  78/150 | train_loss =   409.7490 | val_loss =   436.6627
Epoch  79/150 | train_loss =   408.3552 | val_loss =   435.3805
Epoch  80/150 | train_loss =   407.3160 | val_loss =   434.1412
Epoch  81/150 | train_loss =   406.2620 | val_loss =   432.9500
Epoch  82/150 | train_loss =   405.2479 | val_loss =   431.7956
Epoch  83/150 | train_loss =   404.2175 | val_loss =   430.6895
Epoch  84/150 | train_loss =   403.1359 | val_loss =   429.6197
Epoch  85/150 | train_loss =   402.2821 | val_loss =   428.5877
Epoch  86/150 | train_loss =   401.4995 | val_loss =   427.5805
Epoch  87/150 | train_loss =   400.5427 | val_loss =   426.5998
Epoch  88/150 | train_loss =   399.6107 | val_loss =   425.6661
Epoch  89/150 | train_loss =   398.8798 | val_loss =   424.7623
Epoch  90/150 | train_loss =   398.1709 | val_loss =   423.8937
Epoch  91/150 | train_loss =   397.2868 | val_loss =   423.0385
Epoch  92/150 | train_loss =   396.4887 | val_loss =   422.2139
Epoch  93/150 | train_loss =   395.8716 | val_loss =   421.4314
Epoch  94/150 | train_loss =   395.1128 | val_loss =   420.6363
Epoch  95/150 | train_loss =   394.6416 | val_loss =   419.8792
Epoch  96/150 | train_loss =   393.8950 | val_loss =   419.1548
Epoch  97/150 | train_loss =   393.1262 | val_loss =   418.4472
Epoch  98/150 | train_loss =   392.6709 | val_loss =   417.7422
Epoch  99/150 | train_loss =   392.0872 | val_loss =   417.0604
Epoch 100/150 | train_loss =   391.4323 | val_loss =   416.4110
Epoch 101/150 | train_loss =   390.9872 | val_loss =   415.7863
Epoch 102/150 | train_loss =   390.3880 | val_loss =   415.1346
Epoch 103/150 | train_loss =   389.8094 | val_loss =   414.5305
Epoch 104/150 | train_loss =   389.2223 | val_loss =   413.9384
Epoch 105/150 | train_loss =   388.8190 | val_loss =   413.3597
Epoch 106/150 | train_loss =   388.3430 | val_loss =   412.7855
Epoch 107/150 | train_loss =   387.8501 | val_loss =   412.2395
Epoch 108/150 | train_loss =   387.2959 | val_loss =   411.6899
```

```
Epoch 109/150 | train_loss =   386.7983 | val_loss =   411.1630
Epoch 110/150 | train_loss =   386.4459 | val_loss =   410.6506
Epoch 111/150 | train_loss =   386.0209 | val_loss =   410.1446
Epoch 112/150 | train_loss =   385.7449 | val_loss =   409.6384
Epoch 113/150 | train_loss =   385.0159 | val_loss =   409.1478
Epoch 114/150 | train_loss =   384.7547 | val_loss =   408.6799
Epoch 115/150 | train_loss =   384.3471 | val_loss =   408.2132
Epoch 116/150 | train_loss =   383.8687 | val_loss =   407.7512
Epoch 117/150 | train_loss =   383.5698 | val_loss =   407.3088
Epoch 118/150 | train_loss =   383.2311 | val_loss =   406.8634
Epoch 119/150 | train_loss =   382.8177 | val_loss =   406.4290
Epoch 120/150 | train_loss =   382.4120 | val_loss =   406.0029
Epoch 121/150 | train_loss =   382.0159 | val_loss =   405.5851
Epoch 122/150 | train_loss =   381.9158 | val_loss =   405.1771
Epoch 123/150 | train_loss =   381.4086 | val_loss =   404.7678
Epoch 124/150 | train_loss =   381.0243 | val_loss =   404.3649
Epoch 125/150 | train_loss =   380.7290 | val_loss =   403.9780
Epoch 126/150 | train_loss =   380.3954 | val_loss =   403.5887
Epoch 127/150 | train_loss =   380.0645 | val_loss =   403.2190
Epoch 128/150 | train_loss =   379.8918 | val_loss =   402.8575
Epoch 129/150 | train_loss =   379.5496 | val_loss =   402.4989
Early stopping at epoch 130
```

[50]:
```python
## TODO: Visualize the differences
loss_df = pd.DataFrame(all_losses).dropna(subset = ["mean_val_loss"])
loss_df["batch_size"] = loss_df["batch_size"].astype(str)

plt.figure(figsize = (8, 6))

sns.lineplot(
    data = loss_df,
    x = "epoch",
    y = "mean_val_loss",
    hue = "batch_size",
    palette = "tab10"
)

plt.title("Epoch vs. Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Validation Loss")

plt.xlim(0, loss_df["epoch"].max() + 0.5)

sns.despine()
plt.legend(title = "Batch size")
plt.grid()
plt.show()
```


Epoch vs. Validation Loss
Batch size
4
32
128
1024
Validation Loss
1800
1600
1400
1200
1000
800
600
400
0
20
40
60
80
100
120
Epoch